C Programming Question Bank

Unit: Functions and Recursion

SECTION A: EASY LEVEL (Recall & Basic Application)

A1. Define a function. List any three advantages of using functions (modular programming) in a C program.

A2. Differentiate between a function *declaration* (prototype), a function *definition* and a function *call*. Write one line of code for each, using a function `int square(int n)`.

A3. Write the general syntax of a function definition in C and label its four parts: return type, function name, parameter list and function body.

A4. Every C program contains at least one function. Name it. Who calls it, and what does `return 0;` at its end signify?

A5. Differentiate between *formal parameters* and *actual arguments*. In the code below, identify both:

```
int add(int a, int b) {
    return a + b;
}
...
int s = add(x, 10);
```

A6. Functions are classified into four types based on arguments and return value. Name the four types and write one example prototype for each.

A7. Write a function `void greet(void)` that prints "Welcome to C Functions", and call it from `main()`. Which of the four types is this?

A8. Write a function `int add(int a, int b)` that returns the sum of two integers. In `main()`, read two numbers from the user, call the function and print the result.

A9. Write a function `void printSquare(int n)` that prints the square of `n`. Which type of function is this, and why does it not need a `return` statement with a value?

A10. Write a function `int readNumber(void)` that reads an integer from the keyboard and returns it. Use it twice in `main()` to read two numbers and display their sum.

A11. What does the keyword `void` mean (a) as a return type and (b) inside the parentheses of a parameter list? What error do you get if you write `int x = greet();` where `greet` is a `void` function?

A12. What is the output of the following program?

```
#include <stdio.h>

int cube(int n) {
    return n * n * n;
}

int main() {
    printf("%d", cube(3));
    return 0;
}
```

A13. Write a function `int isEven(int n)` that returns `1` if `n` is even and `0` otherwise. Use it in `main()` to print "Even" or "Odd".

A14. Write a function `int max2(int a, int b)` that returns the larger of two numbers using if-else.

A15. Write a function `float areaCircle(float r)` that returns the area of a circle (take pi = 3.14). Which format specifier is used to print the returned value?

A16. What is a function prototype? Why is it written above `main()` when the function definition is placed below `main()`?

A17. Write a menu-driven calculator in which `add()`, `sub()`, `mul()` and `div()` are four separate functions, each taking two `int` arguments (`add`, `sub`, `mul` return `int`; `div` returns `float`). The operation is chosen in `main()` using a `switch` statement.

A18. Explain *call by value* and *call by reference* in two lines each. Which of the two does C use by default?

A19. What is a pointer? Declare an `int` variable `x = 10` and an `int` pointer `p` that holds its address. Print `x` using `p`. State the role of the `&` and `*` operators in your code.

A20. What is *recursion*? Name the two parts that every correct recursive function must have.

SECTION B: INTERMEDIATE LEVEL (Applied Programs)

B1. Write a recursive function `int fact(int n)` that returns n!. Clearly mark the base case and the recursive case, and trace `fact(4)` step by step: show the order in which the calls are made and the order in which the values are returned.

B2. Predict the output. Explain why every number appears twice and why the second set appears in the opposite order:

```
#include <stdio.h>

void show(int n) {
    if (n == 0)
        return;
    printf("%d ", n);
    show(n - 1);
    printf("%d ", n);
}

int main() {
    show(3);
    return 0;
}
```

B3. Predict the output. Explain why the value of `a` in `main()` does not change even though `x` was modified inside the function:

```
#include <stdio.h>

void change(int x) {
    x = x + 10;
    printf("Inside: %d\n", x);
}

int main() {
    int a = 5;
    change(a);
    printf("Outside: %d\n", a);
    return 0;
}
```

B4. Write a function `void swap(int *a, int *b)` that exchanges two integers using pointers. Show the call from `main()` and explain what each `*` and each `&` in your program does.

B5. A function can return only one value using `return`. Write `void divide(int a, int b, int *quotient, int *remainder)` to hand back *both* the quotient and the remainder of `a / b`. Show how it is called from `main()`.

B6. Write a function `int isPrime(int n)` that returns `1` if `n` is prime and `0` otherwise. Use it in `main()` to print all prime numbers between 1 and 30.

B7. The programmer expected the output 6.50. Predict the actual output, explain why it is not 6.50, and fix the program in two different ways:

```
#include <stdio.h>

float average(int a, int b) {
    return (a + b) / 2;
}

int main() {
    printf("%.2f", average(5, 8));
    return 0;
}
```

B8. Predict the output. What happens to an argument when its type differs from the type of the parameter?

```
#include <stdio.h>

void show(int n) {
    printf("%d ", n);
}

int main() {
    show(3.99);
    show('A');
    show(2 + 3 * 2);
    return 0;
}
```

B9. Write a recursive function `int sumN(int n)` that returns 1 + 2 + ... + n. Trace `sumN(4)` and compare the result with the formula n(n+1)/2.

B10. Write a recursive function `int power(int x, int n)` that returns x raised to n (n >= 0) without using `pow()` or any loop. State the base case.

B11. Write a recursive function `int fib(int n)` that returns the nth Fibonacci number (fib(0) = 0, fib(1) = 1). Draw the call tree of `fib(4)` and count the total number of function calls made.

B12. Write a recursive function `int gcd(int a, int b)` using Euclid's rule (gcd(a, 0) = a, otherwise gcd(a, b) = gcd(b, a % b)). Trace `gcd(48, 18)`.

B13. Write a recursive function `int sumDigits(int n)` that returns the sum of the digits of `n`. Trace it for 1234.

B14. Write a function `void minMax(int a, int b, int *min, int *max)` that stores the smaller number in `*min` and the larger in `*max`. Call it from `main()` and print both.

B15. Predict the output and explain why the two variables behave differently:

```
#include <stdio.h>

void f(int a, int *b) {
    a = a + 1;
    *b = *b + 1;
}

int main() {
    int x = 1, y = 1;
    f(x, &y);
    printf("%d %d", x, y);
    return 0;
}
```

B16. Write two functions: `int isLeap(int year)` and `int daysInMonth(int month, int year)`. The second function must use a `switch` on `month` and must *call* `isLeap()` for February. In `main()`, read a month and a year and print the number of days.

B17. Write both an iterative and a recursive version of the factorial function. Give two differences between the two approaches. Which one uses more memory while running, and why?

B18. Predict the output. Which `x` does `show()` use, and which `x` does `main()` use?

```
#include <stdio.h>

int x = 100;

void show(int x) {
    x = x + 1;
    printf("%d ", x);
}

int main() {
    show(5);
    printf("%d", x);
    return 0;
}
```

SECTION C: HOTS (Higher Order Thinking Skills)

C1. The Failed Swap. A student writes the program below and is surprised that the output is `3 9` and not `9 3`.

```
#include <stdio.h>

void swap(int a, int b) {
    int t = a;
    a = b;
    b = t;
}

int main() {
    int x = 3, y = 9;
    swap(x, y);
    printf("%d %d", x, y);
    return 0;
}
```

(a) Draw the memory of `main()` and of `swap()` (assume `x` and `y` sit at addresses 1000 and 1004, and `a` and `b` at 2000 and 2004). Show the values before and after the three statements inside `swap()`.

(b) Fix the program with the smallest possible change.

(c) "C supports call by reference." Is this statement strictly correct? Justify your answer.

C2. Mystery Function. Predict the output, state what the function computes, and then answer: (i) what is printed for `mystery(0)` and how would you correct it so that it prints `0`? (ii) what is printed if the `printf` is moved *above* the recursive call?

```
#include <stdio.h>

void mystery(int n) {
    if (n == 0)
        return;
    mystery(n / 2);
    printf("%d", n % 2);
}

int main() {
    mystery(10);
    return 0;
}
```

C3. Two Recursive Calls. Predict the exact output without running the code. Then find (i) how many times `printf` executes for `fun(3)`, and (ii) a general formula for `fun(n)`.

```
#include <stdio.h>

void fun(int n) {
    if (n > 0) {
        fun(n - 1);
        printf("%d ", n);
        fun(n - 1);
    }
}

int main() {
    fun(3);
    return 0;
}
```

C4. Returning the Address of a Local Variable. The program below compiles (with a warning) but is dangerous. Explain what happens to `x` when `getValue()` finishes, why `*p` is then unreliable, and give two different ways to fix the design.

```
#include <stdio.h>

int *getValue() {
    int x = 42;
    return &x;
}

int main() {
    int *p = getValue();
    printf("%d", *p);
    return 0;
}
```

C5. Why Does scanf Need &? Using your understanding of pass by value, explain why `scanf("%d", &n);` works but `scanf("%d", n);` does not. What would `scanf` receive in each case, and what could go wrong in the second case?

C6. Variables That Remember. The keyword `static` makes a local variable keep its value between calls. Predict the output of the program. Then explain why `printf("%d %d", next(), next());` is a poor way to test this function (hint: the order in which function arguments are evaluated).

```
#include <stdio.h>

int next(void) {
    static int count = 0;
    count = count + 1;
    return count;
}

int main() {
    printf("%d\n", next());
    printf("%d\n", next());
    printf("%d\n", next());
    return 0;
}
```

C7. Pointer Passed by Value. Predict the output. The function sets its pointer parameter to `NULL`, yet `ptr` in `main()` is unaffected. Explain why, and complete the sentence: "In C, even pass by reference is really pass by _____ of an _____."

```
#include <stdio.h>

void setToNull(int *p) {
    p = NULL;
}

int main() {
    int a = 5;
    int *ptr = &a;
    setToNull(ptr);
    printf("%d", *ptr);
    return 0;
}
```

C8. Mutual Recursion. Predict the output and trace `isEven(4)` call by call. Why is the prototype of `isOdd` mandatory? What happens if `isEven(-2)` is called?

```
#include <stdio.h>

int isOdd(int n);

int isEven(int n) {
    if (n == 0)
        return 1;
    return isOdd(n - 1);
}

int isOdd(int n) {
    if (n == 0)
        return 0;
    return isEven(n - 1);
}

int main() {
    printf("%d %d", isEven(4), isOdd(7));
    return 0;
}
```

C9. The Base Case That Is Not Enough. A student claims `sumTo()` is correct because it "works for 4". Predict what each of the two `printf` lines does and explain the difference. Rewrite the base condition so the function is safe for all inputs.

```
#include <stdio.h>

int sumTo(int n) {
    if (n == 0)
        return 0;
    return n + sumTo(n - 1);
}

int main() {
    printf("%d\n", sumTo(4));
    printf("%d\n", sumTo(-3));
    return 0;
}
```

C10. The Cost of Naive Fibonacci. Draw the complete call tree of `fib(5)` (the function from B11). (a) How many calls are made in total? (b) How many times is `fib(3)` computed, and how many times `fib(2)`? (c) Explain in words why the number of calls grows roughly by a factor of 1.6 every time `n` increases by 1, and suggest one idea to avoid the repeated work.

C11. f() versus f(void). In C (up to C17) the two prototypes below do not mean the same thing. Which of the two `printf` lines fails to compile, and why? What does the empty parameter list `()` actually tell the compiler?

```
#include <stdio.h>

int a() { return 1; }
int b(void) { return 2; }

int main() {
    printf("%d ", a(10));   /* line 1 */
    printf("%d ", b(10));   /* line 2 */
    return 0;
}
```

C12. The Missing Return. The function below returns a value on only one path. Explain what value `check(-5)` gives, why the compiler (with warnings on) complains, and why this kind of bug may "work" on one machine and fail on another. Fix the function.

```
#include <stdio.h>

int check(int n) {
    if (n > 0)
        return 1;
}

int main() {
    printf("%d", check(-5));
    return 0;
}
```

C13. Design Question: Status and Result Together. Write `int safeDivide(int a, int b, int *result)` that returns `1` and stores `a / b` in `*result` when the division is possible, and returns `0` (leaving `*result` untouched) when `b` is zero. Why is this a better design than returning `-1` on error? Show how `main()` should use it.

C14. Recursion and the Stack. Every call creates a new stack frame (parameters, local variables, return address). (a) Draw the stack at the deepest point of `fact(3)`. (b) What is a *stack overflow* and which kind of recursive function causes it? (c) If each frame takes about 32 bytes and the stack is 1 MB (1,048,576 bytes), roughly how many nested calls can be made before it overflows?

SECTION D: DEBUGGING QUESTIONS

For each program, identify the error(s) (compile-time and/or logical / run-time), explain why each one is wrong, and write the corrected code.

D1.

```
#include <stdio.h>

int add(int a, int b)
{
    int sum = a + b
    return sum;
}

int main() {
    int x = 5, y = 7;
    printf("Sum = %d", add(x));
    return 0;
}
```

D2.

```
#include <stdio.h>

int main() {
    float r = 2.5;
    printf("Area = %f", area(r));
    return 0;
}

float area(float r) {
    return 3.14 * r * r;
}
```

D3. The program compiles without any error, but the values are not swapped. Find the bug:

```
#include <stdio.h>

void swap(int *a, int *b) {
    int *t;
    t = a;
    a = b;
    b = t;
}

int main() {
    int x = 3, y = 9;
    swap(&x, &y);
    printf("x=%d y=%d", x, y);
    return 0;
}
```

D4.

```
#include <stdio.h>

void readTwo(int *a, int *b) {
    scanf("%d %d", a, b);
}

int main() {
    int p, q;
    readTwo(p, q);
    printf("%d %d", p, q);
    return 0;
}
```

D5. *(Hint: what stops this recursion?)*

```
#include <stdio.h>

int factorial(int n) {
    return n * factorial(n - 1);
}

int main() {
    printf("%d", factorial(5));
    return 0;
}
```

D6. The program works for 5 but crashes for 0. Find the bug and fix it:

```
#include <stdio.h>

int fact(int n) {
    if (n == 1)
        return 1;
    return n * fact(n - 1);
}

int main() {
    printf("%d\n", fact(5));
    printf("%d\n", fact(0));
    return 0;
}
```

D7. This function should return the sum of digits of 123 (that is, 6) but the program never finishes. Find the bug:

```
#include <stdio.h>

int sumDigits(int n) {
    if (n == 0)
        return 0;
    return n % 10 + sumDigits(n % 10);
}

int main() {
    printf("%d", sumDigits(123));
    return 0;
}
```

D8. The program should print 8 (2 raised to 3) but does not (the actual output is unpredictable). Find the bug:

```
#include <stdio.h>

int power(int x, int n) {
    if (n == 0)
        return 1;
    x * power(x, n - 1);
}

int main() {
    printf("%d", power(2, 3));
    return 0;
}
```

D9.

```
#include <stdio.h>

void square(int n) {
    return n * n;
}

int main() {
    int r = square(5);
    printf("%d", r);
    return 0;
}
```

D10. Find the compile-time error. After fixing it, the program prints 3 although the larger of 2.5 and 3.7 is 3.7. Explain this second (logical) problem and correct it as well:

```
#include <stdio.h>

int max(int a, int b);
{
    if (a > b)
        return a;
    else
        return b;
}

int main() {
    printf("%d", max(2.5, 3.7));
    return 0;
}
```

D11. The program compiles (with a warning) but its behaviour is undefined: it may crash, or it may appear to work. Explain exactly why, and give two different corrections:

```
#include <stdio.h>

void setValue(int *p) {
    *p = 100;
}

int main() {
    int *ptr;
    setValue(ptr);
    printf("%d", *ptr);
    return 0;
}
```

D12. The function is meant to give back the area and perimeter of a rectangle through pointers. Find all the errors:

```
#include <stdio.h>

void areaPerimeter(int l, int b, int *area, int *peri) {
    area = l * b;
    peri = 2 * (l + b);
}

int main() {
    int a, p;
    areaPerimeter(5, 3, &a, &p);
    printf("%d %d", a, p);
    return 0;
}
```

SECTION E: CONSTRAINT-BASED LOGIC QUESTIONS (Design Your Own Function)

These questions do not just ask for a function. They impose a restriction that forces students to think about how parameters, pointers and recursion actually work, rather than plug in a memorised pattern.

E1. Swap Without a Temporary Variable. Write `void swap(int *a, int *b)` in two ways: (i) using only `+` and `-`, (ii) using only the XOR operator `^`. *Restriction:* no third variable. Then call `swap(&x, &x)` with `x = 5` (the same address passed twice). Trace both versions and state the final value of `x`. Why does the ordinary temp-variable version not have this problem?

E2. Maximum Without Decisions. Write `int max2(int a, int b)` using the identity `(a + b + abs(a - b)) / 2` (`abs()` is allowed from `<stdlib.h>`), and then `int max3(int a, int b, int c)` by *calling `max2` only*. *Restriction:* no `if`, no `?:`, no loops, no relational operators inside `max2` and `max3`. Follow-up: for which kinds of inputs can this formula give a wrong answer, and why?

E3. Seconds to HH:MM:SS. Write ONE function that takes a total number of seconds and delivers hours, minutes and seconds to the caller. *Restrictions:* no global variables, no printing inside the function, and the function itself must be the only helper you write. Show the call for 3725 seconds (expected: 1 hour, 2 minutes, 5 seconds).

E4. Multiplication Without the * Operator. Write a recursive function `int multiply(int a, int b)` that multiplies two integers using only `+` and `-`. *Restrictions:* no `*`, no loops, no library

functions. It must also give the correct answer when `b` is negative (for example `multiply(4, -3)` gives `-12`). Explain what goes wrong if you forget to handle negative `b`.

E5. Palindrome Number Using an Accumulator. Check whether a number is a palindrome (121, 1331) by reversing it with recursion. *Restrictions:* no loops, no arrays, no strings. Your reversing function must carry its partial answer in an extra parameter, for example `int rev(int n, int acc)`, called initially as `rev(n, 0)`. Trace `rev(123, 0)` and explain why the extra parameter is needed.

E6. Predict Before You Run. For each of the five calls, predict the value printed *before* compiling. Which functions change the caller's variable and which do not? Explain the surprising result of `f5` using operator precedence.

```
#include <stdio.h>

void f1(int a)  { a = 10; }
void f2(int *a) { *a = 10; }
void f3(int *a) { int b = 10; a = &b; }
void f4(int *a) { (*a)++; *a += 2; }
void f5(int *a) { *a++; }

int main() {
    int x = 1;
    f1(x);  printf("%d ", x);
    f2(&x); printf("%d ", x);
    f3(&x); printf("%d ", x);
    f4(&x); printf("%d ", x);
    f5(&x); printf("%d ", x);
    return 0;
}
```

SECTION F: PLACEMENT-ORIENTED HOTS - APPLIED LOGIC BUILDING

These are framed the way logic-building rounds in placement tests and interviews are framed: a real scenario or constraint, not a "define X" question. All of them are solvable using only what has been taught - functions, pointers, recursion and the Unit 1 conditionals and operators (basic loops only where a question allows). No arrays or strings are needed.

F1. Fast Power (Divide and Conquer). Write a recursive function `long long fastPower(int x, int n)` using the idea: if `n` is even, x^n = (x^(n/2))^2; if `n` is odd, x^n = x * x^(n-1). (a) Count the function calls made by `fastPower(2, 16)` and compare with the simple recursive `power()` of B10. (b) A student writes the even case as `fastPower(x, n/2) * fastPower(x, n/2)`. Explain why the answer stays correct but the speed advantage silently disappears. *(Placement angle: compute a sub-result once and reuse it.)*

F2. Tower of Hanoi. Write `void hanoi(int n, char from, char to, char aux)` that prints each move as "Move disk 1 from A to C". (a) List all moves for `n = 3`. (b) Using the recursion, show that the minimum number of moves is 2^n - 1, and compute it for n = 10 and n = 20. (c) Explain why the order of the three statements (recursive call, print the move, recursive call) cannot be changed.

F3. Count Set Bits Recursively. Write `int countBits(unsigned int n)` that returns the number of 1-bits in `n` using recursion and bitwise operators only (no loops, no `%`). Test with 29 (binary 11101). Follow-up: write a second version using `n & (n - 1)`. What does this expression do to the rightmost 1-bit, and why does this version make fewer calls?

F4. Digital Root. The digital root of a number is found by adding its digits repeatedly until a single digit remains (9875 -> 29 -> 11 -> 2). Write it using two functions: `sumDigits()` (from B13) and a recursive `digitalRoot()` that calls it. Trace `digitalRoot(9875)`. *(Placement angle: breaking a problem into small reusable functions.)*

F5. Quadratic Equation Solver. Write `int solveQuadratic(float a, float b, float c, float *r1, float *r2)` that returns the number of real roots (0, 1 or 2) and stores them through the pointers. If `a == 0` the equation is not quadratic: return `-1` and leave the pointers untouched. `main()` must decide what to print from the returned count. Use `sqrt()` from `<math.h>`. Test with (1, -3, 2), (1, 2, 1), (1, 0, 1) and (0, 2, 3). Explain why the roots cannot be returned with `return`, and why comparing a floating-point discriminant using `== 0` can be risky.

F6. Collatz Sequence (No Loop). Write a recursive function `void collatz(int n)` that prints the sequence: print `n`; stop if `n` is 1; otherwise continue with `n/2` if `n` is even or `3n + 1` if `n` is odd. Show the output for 6. How many steps does 27 take? Discuss: for most recursive functions you can prove the base case is always reached - why is this function different? *(It is an unproven conjecture that every starting number reaches 1.)*

F7. Triangle Pattern Without Loops. (a) Write a recursive function `void stars(int n)` that prints `n` stars on one line. (b) Write a recursive `void triangle(int n)` that prints rows of 1, 2, ..., n stars (for n = 4: 1 star, then 2, 3, 4) by calling `stars()`. (c) Change the position of just one statement so that the triangle is printed upside down (4, 3, 2, 1 stars). Explain why this works.

F8. Perfect Numbers. A number is perfect if the sum of its proper divisors equals the number (6 = 1 + 2 + 3). Write `int isPerfect(int n)` and use it in `main()` to print all perfect numbers below 1000. Follow-up: why is it enough to test divisors only up to `n/2`, and how would you reduce this to about the square root of `n`?

F9. Currency Breakdown. Write `void breakdown(int amount, int *n500, int *n100, int *n50, int *left)` that finds how many Rs. 500, Rs. 100 and Rs. 50 notes are needed (largest first) and what amount is left over. For 1275 the answer is 2, 2, 1 and 25 left. *Restrictions:* the function must not print anything, and `main()` must not do any arithmetic on the notes. *(Placement angle: separating computation from output.)*

F10. The Silent Integer-Division Bug. A programmer wants the percentage of 45 marks out of 60 and gets `0.00`. Trace the expression step by step, using C's precedence and the rule for division of two integers. Fix it in two ways, and then decide what the function should do when `total` is zero.

```
#include <stdio.h>

float percentage(int obtained, int total) {
    return obtained / total * 100;
}

int main() {
    printf("%.2f", percentage(45, 60));
    return 0;
}
```

F11. Predict, Then Justify - Branching Recursion. Without running the code, trace the calls, write the value returned by each call, and give the final output and the total number of calls made:

```
#include <stdio.h>

int g(int n) {
    if (n == 0)
        return 1;
    if (n % 2 == 0)
        return g(n / 2) + 1;
    return g(n - 1) * 2;
}

int main() {
    printf("%d", g(5));
    return 0;
}
```

F12. Trailing Zeros of a Factorial. Write a recursive function `int trailingZeros(int n)` that returns the number of zeros at the end of n! *without computing n!*, using `zeros(n) = n/5 + zeros(n/5)`. (a) Work out the answers for n = 25 and n = 100 by hand. (b) Why is counting factors of 5 enough? (c) Why can't you simply compute `factorial(100)` with an `int` and count the zeros?

SECTION G: PROGRESSIVE FUNCTION PROBLEMS - ONE PATTERN, THREE LEVELS

This section groups classic problems by theme, each taken through three difficulty levels, so students see the same logic pattern become progressively harder instead of jumping between unrelated topics.

G1. Parameter Passing: Swap and Sort

G1.1 (Easy) Write `void swap(int *a, int *b)` using a temporary variable and call it from `main()`.

G1.2 (Intermediate) Write `void sort2(int *a, int *b)` that makes `*a` the smaller and `*b` the larger value (it must call `swap()`). Then write `void sort3(int *a, int *b, int *c)` that arranges three numbers in ascending order by calling `sort2()` three times. Which three pairs, in which order?

G1.3 (HOTS) What happens when `sort2(&x, &x)` is called (same address twice)? Would a swap based on XOR still be safe inside `sort2`? Add a guard to your function so that it also handles a `NULL` pointer without crashing, and explain why checking for `NULL` is the caller's contract in many library functions.

G2. Arithmetic Functions and Return Types

G2.1 (Easy) Write four functions `add`, `sub`, `mul` and `divide` for two integers and print the results for 12 and 5.

G2.2 (Intermediate) Rewrite `divide` so that it returns the exact result (for example 2.4 for 12 and 5) and reports division by zero without crashing: `int divide(int a, int b, float *result)` returns 1 on success and 0 on failure.

G2.3 (HOTS) Write `int safeMultiply(int a, int b, int *result)` for non-negative `a` and `b` that returns 0 and leaves `*result` untouched if `a * b` would exceed `INT_MAX` (from `<limits.h>`), without performing the overflowing multiplication. Explain why checking the product *after* multiplying is not reliable for signed integers. *(A well-known interview gotcha: signed overflow is undefined behaviour in C.)*

G3. The Factorial Family

G3.1 (Easy) Write an iterative function `int factorial(int n)` and print the factorial of a number entered by the user.

G3.2 (Intermediate) Write a recursive `factorial()` and use it to compute nCr = n! / (r! * (n-r)!) in a function `int nCr(int n, int r)`. State the base case clearly.

G3.3 (HOTS) Show that the `int` version gives a wrong result from 13! onwards (12! = 479001600 still fits in a 32-bit `int`). Change the return type to `unsigned long long` and find the first n for which even this overflows (the largest factorial that fits is 20!). Then explain why nCr computed as n!/(r!(n-r)!) can overflow even when the final answer is small, and why the rule nCr(n, r) = nCr(n-1, r-1) + nCr(n-1, r) avoids that problem.

G4. Digit Processing

G4.1 (Easy) Write a function `int countDigits(int n)` that returns how many digits `n` has (use a loop).

G4.2 (Intermediate) Write recursive versions of `countDigits()` and `sumDigits()`. Then write a recursive function that prints the digits of a number in reverse order (1234 prints 4321) and, by moving one statement, in the original order.

G4.3 (HOTS) Write `int isArmstrong(int n)` that works for any number of digits (153, 370, 9474 are Armstrong numbers). Use `countDigits()` and your own recursive `power()` as helpers. Explain why the digit count must be known first, and why using the library `pow()` (which returns a `double`) and then assigning to an `int` can silently give a wrong answer on some systems.

G5. Fibonacci: From Naive to Efficient

G5.1 (Easy) Write the recursive `int fib(int n)` and print the 8th Fibonacci number.

G5.2 (Intermediate) Print the first 10 Fibonacci numbers from `main()` by calling `fib(i)` for each `i`. How many calls does `fib(10)` make in total? (Use a global counter to find out and explain why a global variable is acceptable for this experiment.)

G5.3 (HOTS) Write the efficient version `int fibFast(int n, int a, int b)` that carries the two previous terms as parameters and is called as `fibFast(n, 0, 1)`. Trace `fibFast(5, 0, 1)`. The naive `fib(30)` makes 2,692,537 calls; how many calls does `fibFast(30, 0, 1)` make? Explain what changed: the shape of the recursion (a tree versus a straight line) and the amount of repeated work.

Answer-Key Notes for Instructor (Quick Reference)

Table 1: Key concept being tested

Q. No	Key Concept Being Tested
A1-A6	Why functions exist; declaration vs definition vs call; syntax; formal vs actual parameters; the four types of functions
A7-A11	Writing the four function types; meaning of `void` as return type and as parameter list
A12-A17	Return values; prototypes; modular design (function-based calculator using `switch`)
A18-A19	Call by value vs call by reference; pointer basics: `&` (address-of) and `*` (dereference)
A20	Recursion: base case and recursive case
B1, B9-B13	Recursive thinking: factorial, sum, power, Fibonacci, GCD, digit sum; tracing calls and returns

Q. No	Key Concept Being Tested
B2	Position of a statement before vs after the recursive call controls the order of output
B3, B15, B18	Pass by value: the function works on a copy; mixing value and pointer parameters; parameter shadowing a global variable
B4, B5, B14	Pass by reference using pointers; returning more than one result through pointer parameters
B6, B16	Functions calling other functions; combining Unit 1 logic (`switch`, leap year) with functions
B7, B8	Type conversion of arguments and return values; integer division inside a `float` function
B17	Iteration vs recursion; stack memory used by recursive calls
C1	Why a by-value swap fails; activation records; "C only has pass by value"
C2	Recursion building output in reverse order (decimal to binary); the n = 0 edge case
C3	Two recursive calls around a statement; number of calls = 2^n - 1
C4	Dangling pointer: returning the address of a local variable (undefined behaviour)
C5	Why `scanf` needs `&`: the function needs the address of the variable, not a copy of its value
C6	`static` local variable; unspecified order of evaluation of function arguments
C7	A pointer is itself passed by value; changing the pointer vs changing what it points to
C8	Mutual (indirect) recursion; why a prototype is needed; non-terminating case for negative input
C9	Base case must be reachable for *every* input; stack overflow
C10	Call tree of naive Fibonacci; repeated sub-problems; exponential growth
C11	`f()` (unspecified parameters) vs `f(void)` (no parameters) in C up to C17
C12	Non-`void` function that does not return on every path (undefined behaviour)
C13	Design: return a status code and deliver the result through a pointer
C14	Stack frames, stack overflow, estimating recursion depth
D1	Missing `;`; too few arguments in the call
D2	Function used before declaration (no prototype): conflicting types / implicit declaration
D3	Swapping the local pointer copies instead of the values they point to
D4	Passing `int` where `int *` is expected (missing `&`)
D5	Recursion without a base case
D6	Wrong base case (`n == 1` is never reached for 0); use `n <= 1`
D7	Recursive call does not move towards the base case (`n % 10` instead of `n / 10`)
D8	Result of the recursive call is computed but not returned
D9	`void` function returning a value; using a `void` result
D10	Stray `;` after the function header; `int` parameters truncate `float` arguments
D11	Dereferencing an uninitialised pointer
D12	Missing `*` when assigning through pointer parameters
E1	Arithmetic/XOR swap and the aliasing bug when both pointers refer to the same variable

Q. No	Key Concept Being Tested
E2	Replacing decisions with arithmetic; building a 3-way function from a 2-way function; overflow limits
E3	Several results from one function without globals: pointer parameters, division and remainder
E4	Recursion with a sign-handling edge case; building multiplication from addition
E5	Accumulator parameter in recursion (tail recursion); reversing digits
E6	What a function can and cannot change in the caller; `*a++` vs `(*a)++` precedence
F1	Divide-and-conquer; reusing a sub-result; cost of calling the same recursion twice
F2	Tower of Hanoi: order of the recursive calls; 2^n - 1 moves
F3	Bit counting with recursion; `n & (n - 1)` clears the lowest set bit
F4	Composition of functions; repeated digit sum
F5	Returning a count plus results through pointers; input validation; floating-point comparison
F6	Recursion with branching rule; termination that cannot be proved (Collatz)
F7	Pattern printing with recursion; reordering statements around the recursive call
F8	Divisor-sum function; reducing the search range
F9	Greedy breakdown through pointers; separation of computation from output
F10	Integer division before multiplication in a `float` function; zero-divisor handling
F11	Tracing branching recursion with different returns
F12	Recursion on a mathematical property (factors of 5); avoiding overflow
G1.1-G1.3	Swap, sort of 2 and 3 values through pointers; aliasing and `NULL` guards
G2.1-G2.3	Return types; status code plus result pointer; overflow detection before multiplying
G3.1-G3.3	Iterative and recursive factorial; nCr; overflow limits of `int` and `unsigned long long`; Pascal's rule
G4.1-G4.3	Digit counting, digit sum and reversal; Armstrong numbers; `pow()` returning `double`
G5.1-G5.3	Naive Fibonacci; counting calls; accumulator version that runs in a straight line

Table 2: Expected outputs and answers for predict-the-output, debugging and numeric questions

Q. No	Expected Output / Answer
A12	`27`
B1	`fact(4)` calls `fact(3)`, `fact(2)`, `fact(1)` (base case returns 1); values return as 1, 2, 6, 24
B2	`3 2 1 1 2 3`. The first `printf` runs on the way down; the second runs on the way back up after each call returns
B3	`Inside: 15` then `Outside: 5`. `x` is a copy of `a`
B7	`6.00`. `(5 + 8) / 2` is integer division = 6, converted to float afterwards. Fix: `(a + b) / 2.0` or `(float)(a + b) / 2`
B8	`3 65 8`. 3.99 is truncated to 3; the character 'A' is passed as its code 65; `2 + 3 * 2` = 8

Q. No	Expected Output / Answer
B9, B11-B13	sumN(4) = 10; fib(4) makes 9 calls; gcd(48, 18) -> gcd(18, 12) -> gcd(12, 6) -> gcd(6, 0) = 6; sumDigits(1234) = 10
B15	`1 2`. `a` is a copy, `*b` changes `y` itself
B18	`6 100`. The parameter `x` hides the global `x` inside `show()`
C1	`3 9`. Fix: `swap(int *a, int *b)` with `*a`, `*b` and the call `swap(&x, &y)`
C2	`1010` (binary of 10). `mystery(0)` prints nothing; handle 0 separately to print `0`. With `printf` above the call the output is `0101`
C3	`1 2 1 3 1 2 1`; 7 prints for `fun(3)`; 2^n - 1 prints for `fun(n)`
C4	Undefined behaviour (crashed with segmentation fault on the test system). Fixes: let the caller own the variable and pass its address, or make `xstatic`
C6	`1`, `2`, `3` on separate lines. Argument evaluation order is unspecified, so `1 2` or `2 1` may be printed
C7	`5`. Complete: "pass by *value* of an *address*"
C8	`1 1`. `isEven(4)` `->isOdd(3)` `->isEven(2)` `->isOdd(1)` `->isEven(0)` returns 1. For -2 the argument goes -3, -4, ... and never reaches 0 (stack overflow)
C9	`10` for `sumTo(4)`; `sumTo(-3)` never reaches 0 and crashes. Use `if (n <= 0) return 0;`
C10	15 calls; `fib(3)` is computed 2 times and `fib(2)` 3 times
C11	Line 2 does not compile ("too many arguments to function b"). Line 1 compiles because `a()` leaves the parameters unspecified (up to C17; C23 changed this)
C12	Undefined value (a garbage number was printed on the test system). Add `return 0;` after the `if`
C14	About 1,048,576 / 32 = 32,768 nested calls
D1	Missing `;` after `int sum = a + b`; `add(x)` needs two arguments
D2	`area()` is used before it is declared; add `float area(float r);` above `main()`
D3	Only the local pointers `a`, `b`, `t` are swapped; correct version: `int t = *a; *a = *b; *b = t;`. Output stays `x=3 y=9`
D4	Call must be `readTwo(&p, &q)`
D5	No base case, so recursion never stops (stack overflow). Add `if (n <= 1) return 1;`
D6	`fact(0)` never meets `n == 1`; use `n <= 1`
D7	Must call `sumDigits(n / 10)`
D8	Needs `return x * power(x, n - 1);`; fixed output is `8`
D9	A `void` function cannot return a value and its result cannot be assigned. Change the return type to `int`
D10	Remove the `;` after the header. Then `max(2.5, 3.7)` passes 2 and 3 (truncation) so 3 is printed; use `float` parameters and return type with `%f`
D11	`ptr` is uninitialised, so `*p = 100` writes to an unknown address. Fix: `int v; int *ptr = &v;` or call `setValue(&v)`

Q. No	Expected Output / Answer
D12	Must be `*area = l * b;` and `*peri = 2 * (l + b);`. As written, the pointers are assigned integers and `a`, `p` stay uninitialised
E1	With the same address, both the arithmetic and the XOR version make `x` become `0`. The temp version keeps `5`
E2	Possible overflow in `a + b` and `a - b` for very large values; `abs(INT_MIN)` is undefined
E3	3725 -> 1 hour, 2 minutes, 5 seconds
E4	`multiply(4, -3)` = -12. If a negative `b` is not handled, `b` decreases forever and never reaches 0
E5	`rev(123, 0)` ->`rev(12, 3)` ->`rev(1, 32)` ->`rev(0, 321)` returns 321
E6	`1 10 10 13 13`. `f1` changes a copy; `f2` and `f4` change `x`; `f3` only repoints its local pointer; in `f5`, `*a++` means `*(a++)` (moves the pointer, value unchanged)
F1	`fastPower(2, 16)` makes 6 calls (n = 16, 8, 4, 2, 1, 0) against 17 for the simple version
F2	7 moves for n = 3; 1023 for n = 10; 1,048,575 for n = 20
F3	29 = 11101 has 4 set bits
F4	9875 -> 29 -> 11 -> 2
F5	(1, -3, 2): 2 roots (2 and 1); (1, 2, 1): 1 root (-1); (1, 0, 1): 0 roots; (0, 2, 3): returns -1
F6	6 3 10 5 16 8 4 2 1. 27 takes 111 steps
F8	6, 28, 496
F9	1275 -> two 500s, two 100s, one 50, 25 left
F10	`0.00.45 / 60` is 0 in integer arithmetic, then `0 * 100` = 0. Fix: `100.0f * obtained / total` or cast one operand to `float` first. Return an error value (or use a status/pointer design) when `total` is 0
F11	`8`. g(5) = 2*g(4); g(4) = g(2) + 1; g(2) = g(1) + 1; g(1) = 2*g(0) = 2, so g(2) = 3, g(4) = 4, g(5) = 8. Total calls: 5
F12	25 -> 5 + 1 = 6; 100 -> 20 + 4 = 24
G3.3	13! = 6,227,020,800 is larger than `INT_MAX` (2,147,483,647); 20! is the largest factorial that fits in `unsigned long long`, so 21! overflows
G5.2, G5.3	`fib(10)` makes 177 calls; `fibFast(30, 0, 1)` makes 31 calls

Suggested use: Sections A and B for class tests and quizzes, Section C for viva or bonus questions, Section D as a 10-minute "spot the bug" warm-up at the start of a lab session, Sections E and F for lab assignments and placement-preparation practice, and Section G for graded homework in which students move through the three levels in order.